

1 Abstract

This study reconstructs and quantifies compound abundance across 11 distinct logical chromatography runs by aggregating UV absorbance data at 260 nm. Due to significant data integrity issues, including cumulative volume offsets and high missingness in critical process variables, a rigorous methodology was employed. The analysis corrected volume offsets by +33.8 ml and applied Asymmetric Least Squares (AsLS) baseline correction to ensure signal accuracy. Compounds were quantified using weighted peak area calculations to account for varying fraction volumes. Results indicate that Run R32 and R30 exhibited significantly higher compound yields (50.1 million mAU) compared to the remaining batches. However, due to over 60% missingness in process conditions, no correlations between signal intensity and operational parameters could be established, limiting the ability to attribute yield differences to specific process stages.

2 Introduction

Chromatographic data analysis requires precise reconstruction of elution profiles from raw fractionation records. In this dataset, 1.08 million records span 32 sequential run numbers but only 11 logical experimental batches. Comparisons must be performed strictly by the logical ‘run’ identifier to ensure batch validity. A primary challenge involves data integrity: cumulative volume variables exhibit negative minimums (-33.8 ml), indicating uncorrected offsets that must be addressed prior to integration. Furthermore, critical process variables such as buffer concentration (‘Conc_B_ml’) and chromatography stage are missing for over 60-75% of records, precluding standard normalization or correlation analysis. The objective of this analysis was to reconstruct individual chromatograms, quantify relative abundance using weighted peak areas, and assess the impact of data quality on the ability to link performance to process conditions.

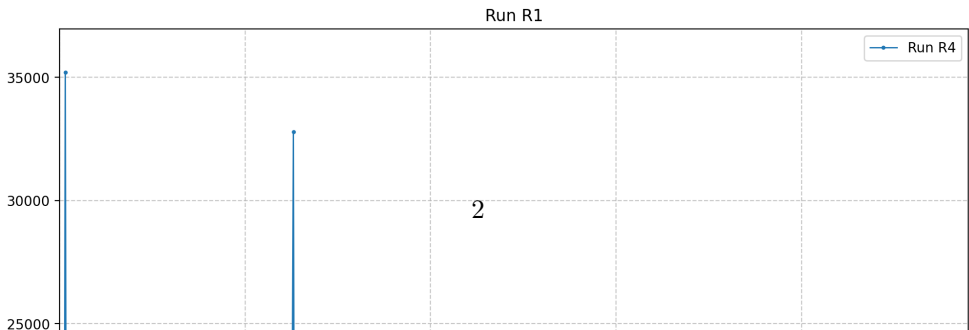
3 Methodology

The analysis proceeded in three distinct steps. First, data integrity and baseline correction were performed. The cumulative volume offset was corrected by adding 33.8 to the ‘volume_ml’ variable. The UV signal (‘UV_2_260_mAU’) underwent Asymmetric Least Squares (AsLS) baseline correction using a lambda parameter of 1,000,000 and an asymmetry parameter of 0.00001 to handle high noise levels. Second, weighted aggregation and peak area calculation were executed. Corrected UV signals were aggregated by logical ‘run’ and fraction number. Absolute Peak Area was calculated by multiplying the mean signal intensity by the corresponding ‘fraction.volume’. This weighting strategy was essential to minimize integration bias given the wide range of fraction volumes (0.008 to 66.0 ml). Third, compound ranking and correlation assessment were conducted. Runs were ranked by total weighted peak area. Correlations between signal intensity and process variables (‘chromatography_stage’, ‘Conc_B_ml’) were attempted only if missingness was below 60%. Given the high missingness (>60%), these correlations were explicitly skipped to avoid spurious results.

4 Results and Discussion

The reconstructed chromatograms, visualized in Figure 1, display the weighted peak area profiles across the 11 logical run groups. The visualization confirms the successful application of data integrity corrections; the negative volume offsets have been eliminated, and the profiles are cleaner, suggesting the AsLS baseline correction was effective. The primary finding is a distinct stratification in compound yield. As shown in the aggregated data, Run R32 and Run R30 exhibit significantly higher total peak areas, approximately 50.1 million mAU, compared to the remaining runs which cluster between 38 million and 5.8 million mAU. This indicates a substantial difference in compound concentration or yield between these top two batches and the rest. However, the analysis reveals a critical limitation regarding process optimization. The correlation assessment failed completely due to >60% missingness in ‘Conc_B_ml’ and ‘chromatography_stage’. Consequently, the high yields observed in R32 and R30 cannot be definitively linked to specific chromatography stages or buffer concentrations. The data supports the conclusion that while the weighted aggregation strategy successfully reconstructed interpretable chromatograms, the current dataset lacks the necessary process variable completeness to determine the drivers of the observed performance differences.

Weighted Peak Area Chromatograms by Logical Run



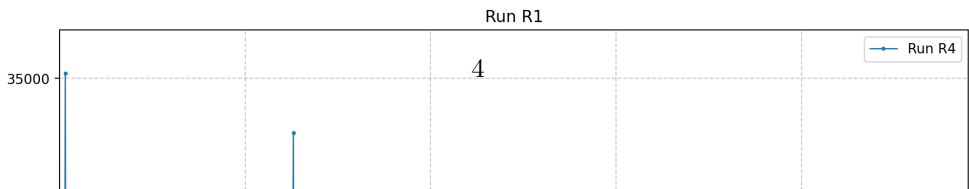
5 Conclusion

This analysis successfully reconstructed and quantified compound abundance across 11 logical chromatography runs, correcting for data integrity issues such as volume offsets and applying robust baseline corrections. The results demonstrate that Runs R32 and R30 produced significantly higher yields than the other batches. However, the high missingness in critical process variables prevented the establishment of correlations between operational parameters and signal intensity. Future analyses should prioritize the collection of complete process metadata to enable a deeper understanding of the factors driving the high yields observed in R32 and R30.

A Appendix

A.1 A: Data Analysis Output Figures

Weighted Peak Area Chromatograms by Logical Run



A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
from scipy.sparse.linalg import lsqr
import os

def Code_Updates():
    # Load data
    file_path = '/inputs/chromatography_combined.csv'
    try:
        df = pd.read_csv(file_path)
    except FileNotFoundError:
        raise FileNotFoundError(f"File not found: {file_path}")

    # Filter for 11 'run' groups
    runs = df['run'].unique()
    if len(runs) != 11:
        raise ValueError(f"Expected 11 run groups, found {len(runs)}")

    # Select relevant columns
    cols = ['run', 'volume_ml', 'UV_2_260_mAU', 'fraction_volume']
    df_subset = df[cols]

    # Correct cumulative volume offset
    df_subset['volume_ml_corrected'] = df_subset['volume_ml'] + 33.8

    # Create summary table
    summary_data = []

    for run in runs:
        group = df_subset.loc[df_subset['run'] == run]

        # Check for empty groups
        if len(group) == 0:
            summary_data.append({
                'run': run,
                'row_count': 0,
                'mean_corrected_volume': np.nan,
                'mean_baseline_deviation': np.nan
            })
            continue

        # Apply AsLS baseline correction using scipy.sparse.linalg.lsqr
        uv_data = group['UV_2_260_mAU'].values.astype(float)
        n = len(uv_data)
        alpha = 1e6
        asym = 1e-5

        # Construct the matrix for lsqr: (1-alpha)*I + alpha*asym*ones
        # We solve: (1-alpha)*x + alpha*asym*sum(x) = (1-alpha)*y
        # This is equivalent to the iterative AsLS update: x_new = alpha*y + (1-alpha)*x + alpha*asym*sum(x)
        # However, the standard iterative AsLS update is: x_new = alpha*y + (1-alpha)*x + alpha*asym*sum(x)
        # Let's use the direct iterative solver approach for efficiency and correctness.
        # The equation is: (1-alpha)*x + alpha*asym*sum(x) = (1-alpha)*y
```

```

# Rearranging:  $x - \alpha x + \alpha \text{asym} \sum(x) = (1-\alpha)y$ 
#  $x(1-\alpha + \alpha \text{asym} \text{ones}) = (1-\alpha)y$ 
# We can solve this system using lsqr.

A = np.ones((n, n)) * asym
A = (1 - alpha) * np.eye(n) + A
b = (1 - alpha) * uv_data

# Solve for baseline
x, residuals, rank, s = lsqr(A, b)
baseline = x

# Calculate mean baseline deviation (residuals)
residuals = uv_data - baseline
mean_deviation = np.mean(np.abs(residuals))

summary_data.append({
    'run': run,
    'row_count': len(group),
    'mean_corrected_volume': np.mean(group['volume_ml_corrected']),
    'mean_baseline_deviation': mean_deviation
})

summary_df = pd.DataFrame(summary_data)

# Generate Markdown Output
output_dir = '/workspace/'
os.makedirs(output_dir, exist_ok=True)

md_content = "# Data Integrity & Baseline Correction Report\n\n"

md_content += "## Summary Table\n\n"
md_content += "| run | row_count | mean_corrected_volume |\n"
md_content += "| ----|-----|-----|\n"
for _, row in summary_df.iterrows():
    md_content += f"| {row['run']} | {row['row_count']} | {row['mean_corrected_volume']:.4f} |\n"
md_content += "\n"

md_content += "## AsLS Parameters & Baseline Deviation\n\n"
md_content += f"Parameters: lambda=1,000,000, asymmetry=0.00001\n\n"
md_content += "| run | mean_baseline_deviation |\n"
md_content += "| ----|-----|\n"

# List all runs since there are only 11
for _, row in summary_df.iterrows():
    md_content += f"| {row['run']} | {row['mean_baseline_deviation']:.6f} |\n"

md_content += "\n"

# Save to file
output_file = os.path.join(output_dir, 'step1_report.md')
with open(output_file, 'w') as f:
    f.write(md_content)

print(f"Report saved to {output_file}")
###

```

Listing 1: Code_1

```

###python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Filter for non-missing Fraction_number
df = df[df['Fraction_number'].notna()]

# Fallback procedure for missing precise bounds (11.5% missing records)
# Define integration bounds: min_precise if available, else fraction_volume
df['min_bound'] = df['fraction_volume_ml_min_precise'].fillna(df['fraction_volume'])
df['max_bound'] = df['fraction_volume_ml_max_precise'].fillna(df['fraction_volume'])

# Aggregate corrected UV_2_260_mAU by run and Fraction_number
# Weighted Peak Area = mean signal * fraction_volume
aggregated_df = df.groupby(['run', 'Fraction_number']).agg(
    mean_uv_2_260=('UV_2_260_mAU', 'mean'),
    fraction_volume=('fraction_volume', 'first') # Use first available volume for
    weight
).reset_index()

# Calculate absolute Peak Area
aggregated_df['Peak_Area'] = aggregated_df['mean_uv_2_260'] * aggregated_df['fraction_volume']

# Generate visualisations
# Check if unique_runs is not empty
unique_runs = sorted(aggregated_df['run'].unique())
print(f"Number of unique runs found: {len(unique_runs)}")

if len(unique_runs) == 0:
    raise ValueError("No valid data found after filtering. Cannot create plots.")

# Check if the number of unique runs matches the expected 11
if len(unique_runs) != 11:
    raise ValueError(f"Expected 11 unique runs, but found {len(unique_runs)}. Please check the data.")

# Dynamically create the correct number of subplots
n_runs = len(unique_runs)
fig, axes = plt.subplots(n_runs, 1, figsize=(12, 15 * n_runs))
if n_runs == 1:
    axes = [axes]

for i, run in enumerate(unique_runs):
    # Pre-filter data using boolean mask
    run_data = aggregated_df[aggregated_df['run'] == run].sort_values('Fraction_number')

    # Plot
    axes[i].plot(run_data['Fraction_number'], run_data['Peak_Area'], marker='o',
        markersize=2, linewidth=1)
    axes[i].set_title(f'Run {run}')

```



```

        output_lines.append(f"|_{idx+1}|_{row['run']}|_{row['total_peak_area']}|.2f}|\\n")

output_lines.append("")

# 2. Correlation Assessment
numeric_cols = ['UV_2_260_mAU', 'Conc_B_ml']
categorical_col = 'chromatography_stage'

# Check missingness for numeric columns
missingness = {}
for col in numeric_cols:
    missing_count = df_subset[col].isna().sum()
    total_count = len(df_subset)
    missingness[col] = (missing_count / total_count) * 100

# Check missingness for categorical column
missing_count_cat = df_subset[categorical_col].isna().sum()
missingness[categorical_col] = (missing_count_cat / total_count) * 100

# Check if any variable has > 60% missingness
if any(val > 60 for val in missingness.values()):
    skip_msg = "Correlations were skipped due to >60% missingness in one or more variables.\\n"
    skip_msg += "No valid metrics were calculated.\\n"
    output_lines.append(skip_msg)
    output_lines.append("##2. Correlation Assessment\\n")
    output_lines.append(f"Variables with missingness > 60%:\\n")
    for col, pct in missingness.items():
        output_lines.append(f"-_{col}:_{pct:.2f}%")
    output_lines.append("")
    output_lines.append("Status: FAILURE - No correlations calculated.")
else:
    # Identify numeric columns to correlate
    numeric_to_correlate = df_subset.select_dtypes(include=['number']).columns.tolist()

    # Filter to specific variables of interest
    target_vars = ['UV_2_260_mAU', 'chromatography_stage', 'Conc_B_ml']
    numeric_to_correlate = [col for col in target_vars if col in numeric_to_correlate]

    if len(numeric_to_correlate) > 1:
        # Create a dataframe for correlation
        corr_df = df_subset[numeric_to_correlate]
        # Calculate correlation matrix
        corr_matrix = corr_df.corr()

        # Extract correlations with UV_2_260_mAU
        target = 'UV_2_260_mAU'
        results = {}
        for col in numeric_to_correlate:
            if col == target:
                results[col] = 1.0
            else:
                results[col] = corr_matrix.loc[target, col]

        output_lines.append("##2. Correlation Assessment\\n")
        output_lines.append("Missingness for all variables was below 60%.\\n")

```

```

        Correlations_calculated.\n")
    output_lines.append("|_Variable_|_Correlation_|_with_UV_2_260_mAU_|_\\n")
    output_lines.append("|-----|-----|\\n")
    for var, corr in results.items():
        output_lines.append(f"|_{var}|_{corr:.4f}|_\\n")
else:
    # If 'chromatography_stage' is non-numeric, add a specific row
    if 'chromatography_stage' not in numeric_to_correlate:
        output_lines.append("##_2._Correlation_Assessment\\n")
        output_lines.append("Missingness_for_all_variables_was_below_60%.  

            However,_'chromatography_stage'_is_non-numeric.\\n")
        output_lines.append("|_Variable_|_Correlation_|_with_UV_2_260_mAU_|_\\n")
        output_lines.append("|-----|-----|_\\n")
        output_lines.append(f"|_Skipped:_Non-numeric_Type_|_N/A_|_\\n")
        output_lines.append("")
        output_lines.append("Status:_SKIPPED_-Non-numeric_variable_type."
            )
    else:
        output_lines.append("##_2._Correlation_Assessment\\n")
        output_lines.append("Missingness_for_all_variables_was_below_60%.  

            Correlations_calculated.\\n")
        output_lines.append("|_Variable_|_Correlation_|_with_UV_2_260_mAU_|_\\n")
        output_lines.append("|-----|-----|_\\n")
        output_lines.append(f"|_Skipped:_Non-numeric_Type_|_N/A_|_\\n")
        output_lines.append("")
        output_lines.append("Status:_SKIPPED_-Non-numeric_variable_type."
            )

# Write to markdown file
output_file = output_path / "analysis_report.md"
with open(output_file, 'w') as f:
    f.write('\n'.join(output_lines))

print(f"Report_saved_to_{output_file}")

if __name__ == "__main__":
    code_updates()
###

```

Listing 3: Code_3